

山东大学
SHANDONG UNIVERSITY

密码分析学第四次实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：November 1, 2026

Contents

1 组员分工 **2**

2 实验环境 **2**

3 实验原理 **2**
3.1 DDT 与 LAT 2
3.1.1 DDT 2
3.1.2 LAT 3
3.2 S 盒传播的统一建模约定 3
3.3 one-hot 法 3
3.4 逐点排除法 4
3.5 逻辑化简法 5
3.6 凸包法 7
3.7 全解枚举与等价性验证 8

4 实验内容 **9**
4.1 任务一：DDT 与 LAT 的建模及求解 9
4.1.1 one-hot 法 11
4.1.2 逐点排除法 12
4.1.3 逻辑化简法 13
4.1.4 凸包法 15
4.1.5 四种方法的结果对比 17

1 组员分工

- 邱泽辉：待填写
- 江李阳：待填写
- 王健一：待填写

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS、WSL Arch Linux，Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。
- 求解与建模工具：Gurobi 及其 Python 接口 `gurobipy` 13.0.3、PyEDA 0.29.0（Espresso 逻辑化简）、SageMath 10.9（凸包构造）。
- Python 依赖管理：`uv`。

3 实验原理

3.1 DDT 与 LAT

根据课程所学的差分分析和线性分析，这里简单复述一下 S 盒的差分分布表 (DDT) 和线性分布表 (LAT)。

3.1.1 DDT

对于一个 $m \times n$ 的 S 盒，即输入 m 比特、输出 n 比特的映射 $S : \mathbb{F}_2^m \to \mathbb{F}_2^n$，记满足输入差分 α、输出差分 β 的输入集合为

$$IN_S(\alpha, \beta) = \{X \in \mathbb{F}_2^m : S(X) \oplus S(X \oplus \alpha) = \beta\}.$$

DDT 中的表项为该集合的元素个数，即

$$\mathrm{DDT}[\alpha][\beta] = \#IN_S(\alpha, \beta).$$

可见这是一张 $2^m \times 2^n$ 的表，构建时可以先遍历输入差分 $\alpha \in \mathbb{F}_2^m$ 共 2^m 种可能，内层循环遍历输入 $X \in \mathbb{F}_2^m$ 共 2^m 种可能，根据计算出的输出差分更新表项。因此，构建 DDT 需要检查 $2^m \times 2^m = 2^{2m}$ 组 (α, X) 对应的差分传播。

3.1.2 LAT

对于同一个 $m \times n$ 的 S 盒，LAT 是一个 $2^m \times 2^n$ 的表，其中第 α 行第 β 列的表项为

$$\mathrm{LAT}[\alpha][\beta] = \#\{x \in \mathbb{F}_2^m \mid \alpha \cdot x \oplus \beta \cdot S(x) = 0\} - 2^{m-1}.$$

这里 $\cdot$ 表示二进制向量的模 2 内积。该表项是输入、输出掩码为 α, β 时，使线性表达式 $\alpha \cdot x \oplus \beta \cdot S(x)$ 等于 0 的输入个数与 2^{m-1} 的差，其值与输入总数 2^m 的比值恰好为线性近似的偏差：

$$\epsilon(\alpha, \beta) = \Pr[\alpha \cdot x = \beta \cdot S(x)] - \frac{1}{2} = \frac{\mathrm{LAT}[\alpha][\beta]}{2^m}.$$

其中概率是在输入 x 均匀分布于 $\mathbb{F}_2^m$ 时计算的。构建该表时，可以先遍历输入掩码 $\alpha \in \mathbb{F}_2^m$，内层遍历输出掩码 $\beta \in \mathbb{F}_2^n$，最内层遍历输入 $x \in \mathbb{F}_2^m$，统计线性表达式为 0 的输入个数。因此，共需要检查 $2^m \times 2^n \times 2^m = 2^{2m+n}$ 组 (α, β, x) 对应的线性表达式。

3.2 S 盒传播的统一建模约定

本实验使用 4 比特输入、4 比特输出的 S 盒。对于输入、输出差分对或掩码对 (a, b)，将两者分别按最高位到最低位展开，再按输入在前、输出在后的顺序拼接为

$$\mathrm{enc}(a, b) = (a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0) \in \{0,1\}^8,$$

其中 $a = \sum_{i=0}^{3} 2^i a_i$，$b = \sum_{i=0}^{3} 2^i b_i$。据此，分别定义差分传播和线性掩码传播的合法点集为

$$V_{\mathrm{D}} = \{\mathrm{enc}(a, b) \mid a, b \in \{0, \ldots, 15\},\ \mathrm{DDT}[a][b] > 0\},$$
$$V_{\mathrm{L}} = \{\mathrm{enc}(a, b) \mid a, b \in \{0, \ldots, 15\},\ \mathrm{LAT}[a][b] \neq 0\}.$$

本报告的 DDT 与 LAT 均按输入在行、输出在列存储；讲义中的传播矩阵按输出在行、输入在列记作 $M[b, a]$ 或 $C[b, a]$。对于本实验的四位 S 盒，两种记法的对应关系为

$$M[b, a] = \frac{\mathrm{DDT}[a][b]}{16}, \qquad C[b, a] = \frac{\mathrm{LAT}[a][b]}{8}.$$

其中 C 为相关度矩阵，LAT 沿用前文的计数减半定义。矩阵转置的存储约定和非零缩放均不改变同一对 (a, b) 是否属于合法集合，但提取点集时必须保持输入、输出顺序一致。

LAT 中的负数表示负偏差，仍属于非零相关的合法掩码对，因此构造 V_{L} 时不能只保留正数表项。两个集合均保留 $(a, b) = (0, 0)$ 对应的全零向量，以描述未活跃 S 盒的传播。

下面统一用 V 表示 V_{D} 或 V_{L}，用 $z = (z_1, \ldots, z_8)$ 表示待约束的二进制变量。建模要求是：约束在这八个原始变量上的可行点集恰好为 V；如果引入辅助变量，则取全部可行解在 z 上的投影。四种方法分别对两个集合应用相同的构造规则，仅刻画传播的可行性，不额外编码具体的概率或相关度权重。

3.3 one-hot 法

基本原理 One-hot（独热编码）法按讲义中的三个步骤建立模型：先列出合法点并为每个点引入一个选择变量，再要求恰好选中一个点，最后按坐标恢复被选中点的值。该方法直接使用点集，不需要先计算凸包或进行逻辑化简。

数学建模 沿用第3.2节的统一记号，将合法点集写为

$$V = \{v^{(1)}, \dots, v^{(N)}\}, \qquad v^{(i)} = (v_1^{(i)}, \dots, v_8^{(i)}),$$

其中 $N = |V|$。将各点作为列向量排成常数矩阵 $A = (v^{(1)}, \dots, v^{(N)})$，并令 $\lambda = (\lambda_1, \dots, \lambda_N)^{\mathsf{T}}$，则唯一选择和坐标恢复分别写为

$$\mathbf{1}^{\mathsf{T}}\lambda = 1, \qquad z = A\lambda.$$

逐坐标展开，并明确原始变量与选择变量的二进制取值，得到模型

$$\boxed{\begin{aligned} &\sum_{i=1}^{N} \lambda_i = 1, \\ &z_j = \sum_{i=1}^{N} v_j^{(i)} \lambda_i, \quad j = 1, \dots, 8, \\ &\lambda_i \in \{0,1\}, \qquad i = 1, \dots, N, \\ &z_j \in \{0,1\}, \qquad j = 1, \dots, 8. \end{aligned}} \tag{1}$$

其中，第一项约束保证唯一选择，第二项约束负责逐坐标恢复该合法点。由于 $v_j^{(i)}$ 均为已知的 0/1 常数，第 j 个坐标也可稀疏地写成

$$z_j = \sum_{i:\, v_j^{(i)} = 1} \lambda_i.$$

因此，程序只需累加在该坐标取 1 的点对应的选择变量；若没有这样的点，则该坐标等于 0。所有等式都不含变量之间的乘积，仍为线性约束。模型中的原始变量与辅助选择变量应分别统计，不能把选择变量数当作模型的总变量数。

正确性分析 对于模型的任意可行解，由 λ_i 的二进制取值及其和为 1 可知，存在唯一的 k 使得 $\lambda_k = 1$，其余选择变量均为 0。代入坐标等式，得到 $z_j = v_j^{(k)}$，从而 $z = v^{(k)} \in V$。因此，模型不会引入非法点。

反过来，任取合法点 $v^{(k)} \in V$，令 $z = v^{(k)}$、$\lambda_k = 1$，其余 $\lambda_i = 0$，即可满足全部约束。因此，每个合法点都能由模型表示。综上，

$$\left\{ z \in \{0,1\}^8 \mid \exists \lambda \in \{0,1\}^N \text{满足式(1)} \right\} = V.$$

用于差分与线性传播 差分建模时取 $V = V_{\mathrm{D}}$，每个选择变量对应一个 DDT 非零位置；线性建模时取 $V = V_{\mathrm{L}}$，每个选择变量对应一个 LAT 非零位置。也可以直接用 (a, b) 给选择变量编号，记作 $\lambda_{(a,b)}$，从而与原表中的位置一一对应。两类模型分别建立各自的选择变量，坐标约束形式保持一致；全零点的选择变量也需要保留，以表示未活跃 S 盒。表项的具体数值不作为这里的优化目标，枚举结果应覆盖全部合法点。

3.4 逐点排除法

基本原理 逐点排除法从所有八位二进制向量出发，对每个非法点建立一条只排除该点的不等式。将这些不等式同时加入模型后，所有非法点都被排除，而合法点全部保留。

数学建模 设非法点集为

$$U = \{0,1\}^8 \setminus V.$$

对于任意 $u = (u_1, \ldots, u_8) \in U$，定义 z 与 u 的 Hamming 距离

$$d_u(z) = \sum_{j:u_j=0} z_j + \sum_{j:u_j=1} (1 - z_j).$$

当 $u_j = 0$ 时，z_j 恰好表示该位是否不同；当 $u_j = 1$ 时，$1 - z_j$ 恰好表示该位是否不同。因此，在二进制变量上，$d_u(z)$ 等于两向量不同坐标的数量。

要求 z 至少有一位不同于每个非法点，可得模型

$$\boxed{\begin{aligned} &\sum_{j:u_j=0} z_j + \sum_{j:u_j=1} (1 - z_j) \geq 1, \quad u \in U, \\ &z_j \in \{0,1\}, \qquad\qquad\qquad\qquad j = 1, \ldots, 8. \end{aligned}}$$

由于 u 是已知的常量向量，每条不等式也可以整理为标准线性形式

$$\sum_{j:u_j=0} z_j - \sum_{j:u_j=1} z_j \geq 1 - \sum_{j=1}^{8} u_j, \qquad u \in U.$$

例如，输入为 0、输出为 1 对应的点为 $u = 0000\,0001$。对于本实验的置换 S 盒，该点既不属于差分合法集合，也不属于线性合法集合，其排除约束为

$$z_1 + z_2 + z_3 + z_4 + z_5 + z_6 + z_7 + (1 - z_8) \geq 1.$$

将 0000 0001 代入，左端为 0；改变任意一位，左端便至少为 1。这说明约束由非法点的坐标直接生成，不需要求解器先寻找其系数。

正确性分析 对于固定的非法点 u，若 $z = u$，则 $d_u(z) = 0$，违反对应约束；若 $z \neq u$，则至少一个坐标不同，从而 $d_u(z) \geq 1$。因此，一条约束在二进制空间中恰好排除其对应的非法点。

若 $z \in V$，则 z 不同于每个 $u \in U$，故满足全部不等式。若 $z \notin V$，则 $z \in U$，模型中存在针对 $u = z$ 的约束，该约束必然被违反。由此可知，模型的二进制可行点集恰好为 V。

用于差分与线性传播 差分建模时，将 DDT 零位置对应的八位向量作为 U；线性建模时，将 LAT 零位置对应的八位向量作为 U。两种情况下均只使用原始变量 z，不需要额外的选择变量。

3.5 逻辑化简法

基本原理 逐点排除法对每个非法点分别建立约束，而非法点之间可能共享部分坐标。逻辑化简法将这些点作为布尔函数的取值为 1 的输入，通过 Espresso 等工具合并为 cube，再对每个 cube 建立一条排除约束。一条这样的约束可以同时排除该 cube 覆盖的全部非法点。

非法点的布尔表示 设 $U = \{0,1\}^8 \setminus V$，定义非法点指示函数

$$f_{\text{bad}}(z) = \begin{cases} 1, & z \in U, \\ 0, & z \in V. \end{cases}$$

将该函数的真值表或积之和表达式交给逻辑化简工具，得到 cube 集合 $\mathcal{C}$。每个 cube 写成

$$c = (c_1, \ldots, c_8) \in \{0, 1, -\}^8,$$

其中 0、1 表示该坐标固定，– 表示该坐标不限。记

$$I_0(c) = \{j \mid c_j = 0\}, \qquad I_1(c) = \{j \mid c_j = 1\},$$

则该 cube 覆盖的二进制点集为

$$B(c) = \{z \in \{0,1\}^8 \mid z_j = 0 \ (j \in I_0(c)),\ z_j = 1 \ (j \in I_1(c))\}.$$

精确化简要求覆盖关系满足

$$\bigcup_{c \in \mathcal{C}} B(c) = U. \tag{2}$$

该等式既要求所有非法点都被覆盖，也要求没有合法点落入任何 cube。构造输入真值表时，合法点应明确标为 0，不能将其当作无关项；cube 中的 – 仅表示化简后该坐标不受限制。

讲义使用 `espresso-logic` 读取 PLA 格式的非法点列表：本实验应声明八个输入、一个输出，输入标签按 $a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0$ 排列，每个非法点的输出标为 1，未列出的点按对应 PLA 输入模式解释为输出 0，不能当作任意取值。读取化简结果时，逐行提取输出为 1 的 cube；其中的`-`表示该位不限，`.p` 字段表示输出的 cube 数量。无论采用讲义中的`-Dexact` 方式，还是通过其他接口调用 Espresso，后续线性化都依赖式(2)所要求的精确覆盖关系。

cube 的线性约束 为了排除 $B(c)$，要求 z 至少有一个固定坐标不同于 c，得到

$$\boxed{\begin{aligned} &\sum_{j \in I_0(c)} z_j + \sum_{j \in I_1(c)} (1 - z_j) \geq 1, \quad c \in \mathcal{C}, \\ &z_j \in \{0,1\}, \qquad\qquad\qquad\qquad j = 1, \ldots, 8. \end{aligned}} \tag{3}$$

整理常数项后，每条约束可写为

$$\sum_{j \in I_0(c)} z_j - \sum_{j \in I_1(c)} z_j \geq 1 - |I_1(c)|.$$

cube 中取值为 – 的坐标不出现在不等式中；若一个 cube 的全部坐标都固定，上式便退化为逐点排除约束。

正确性分析 对于给定的 c，若 $z \in B(c)$，则所有固定坐标均与 c 一致，式(3)中对应排除不等式的左端为 0，从而该点被排除。若 $z \notin B(c)$，则至少一个固定坐标不同，左端至少为 1，满足对应不等式。因此，每条不等式恰好排除一个 cube 覆盖的点集。

结合式(2)可知，全部约束恰好排除 U，留下的二进制可行点集就是 V。用于差分传播时，对 DDT 零位置化简；用于线性传播时，对 LAT 零位置化简。化简后的约束只涉及原始变量，但启发式化简并不保证得到全局最少的 cube，仍需核对覆盖关系是否精确。

3.6 凸包法

基本原理 凸包法把合法的二进制点集 V 看作实数空间中的点集，先求出其凸包，再用凸包的线性不等式和等式描述传播关系。由于最终仍要求原始变量取二进制值，完整 H 表示在这些变量上的可行点集恰好为 V。

凸包与 H 表示 设 $V = \{v^{(1)}, \dots, v^{(N)}\} \subseteq \{0,1\}^8$，其凸包定义为

$$P = \mathrm{conv}(V) = \left\{ \sum_{i=1}^{N} \theta_i v^{(i)} \;\middle|\; \theta_i \geq 0,\ \sum_{i=1}^{N} \theta_i = 1 \right\}.$$

其中 θ_i 为非负实数，此处仅用于给出凸包定义。将合法点作为顶点输入 SageMath 等工具，可得到 P 的 H 表示：

$$P = \left\{ z \in \mathbb{R}^8 \;\middle|\; \begin{array}{ll} (a^{(k)})^{\mathsf{T}} z + \beta_k \geq 0, & k \in K, \\ (e^{(\ell)})^{\mathsf{T}} z + \gamma_\ell = 0, & \ell \in E \end{array} \right\}.$$

这里 K、E 分别为不等式和等式的索引集合，系数均为已知常数。当凸包不是满维时，H 表示可能包含等式，读取结果时需要保留这些等式。为避免系数舍入改变可行点集，可使用精确有理数计算，并通过乘以正的公分母将约束整理成整数系数形式。

讲义通过 `Polyhedron(vertices=V)` 从点集构造多面体，并用 `inequalities_list()` 读取系数。本实验每个点应为八位向量，因此输出的一行系数写成 $q = (c_0, c_1, \dots, c_8)$，对应

$$h_q(z) = c_0 + \sum_{j=1}^{8} c_j z_j \geq 0.$$

其中第一项 c_0 是常数项，不是第一个坐标的系数。等式系数需要另外读取，并以 $h_q(z) = 0$ 加入模型；不能仅提取不等式而丢弃等式条件。

由此得到完整凸包模型

$$\boxed{\begin{array}{ll} (a^{(k)})^{\mathsf{T}} z + \beta_k \geq 0, & k \in K, \\ (e^{(\ell)})^{\mathsf{T}} z + \gamma_\ell = 0, & \ell \in E, \\ z_j \in \{0,1\}, & j = 1, \dots, 8. \end{array}} \tag{4}$$

完整模型的正确性 首先，任意 $v \in V$ 都属于其凸包，故满足全部 H 表示约束。反过来，若二进制向量 $z \in P$，则存在非负系数 θ_i，使 $z = \sum_i \theta_i v^{(i)}$ 且 $\sum_i \theta_i = 1$。

当 $z_j = 0$ 时，所有 $\theta_i > 0$ 的点在第 j 个坐标都必须为 0；当 $z_j = 1$ 时，这些点在第 j 个坐标都必须为 1，否则加权平均会小于 1。因此，所有正系数对应的点都与 z 逐坐标相同，必有 $z \in V$。于是

$$P \cap \{0,1\}^8 = V,$$

说明式(4)恰好描述合法点集。

基于非法点覆盖的贪心筛选 完整 H 表示可能包含较多约束，而本实验只要求二进制可行点准确。因此，可以按照每条不等式排除非法点的能力进行筛选。设 $U = \{0,1\}^8 \setminus V$，对每个 $k \in K$ 定义

$$R_k = \{u \in U \mid (a^{(k)})^{\mathsf{T}} u + \beta_k < 0\}.$$

保留全部等式，并记等式已经排除的非法点集为

$$R_E = \{u \in U \mid \exists \ell \in E,\ (e^{(\ell)})^{\mathsf{T}} u + \gamma_\ell \neq 0\}.$$

初始化尚未排除的点集 $U_0 = U \setminus R_E$，选中的不等式索引集合 $K_0 = \varnothing$。只要 $U_t \neq \varnothing$，便选择

$$k_t \in \operatorname*{arg\,max}_{k \in K \setminus K_t} |R_k \cap U_t|,$$

并更新

$$K_{t+1} = K_t \cup \{k_t\}, \qquad U_{t+1} = U_t \setminus R_{k_t}.$$

若多个候选约束覆盖相同数量的点，可约定按固定的系数顺序选择，以便复现筛选过程。由于完整 H 表示能够排除每个非法二进制点，只要仍有点未被排除，就应存在能够覆盖其中至少一点的候选约束。

这对应讲义中的三个列表：`remaining` 保存尚未排除的非法点，`pool` 保存未选中的候选不等式，`chosen` 保存已经保留的不等式。选中一条约束后，只从 `remaining` 中删除违反它的点，仍满足它的点继续留在列表中；同时将该约束从候选列表移至保留列表。若剩余点非空而最佳候选不能再排除任何点，应检查 H 表示、位序或系数读取，不能继续把该过程当作有效约简。

当剩余点集为空时，记最终选中的索引集合为 K_*，保留的模型为

$$\begin{array}{ll} (a^{(k)})^{\mathsf{T}} z + \beta_k \geq 0, & k \in K_*, \\ (e^{(\ell)})^{\mathsf{T}} z + \gamma_\ell = 0, & \ell \in E, \\ z_j \in \{0,1\}, & j = 1, \ldots, 8. \end{array}$$

所有保留的约束均来自原 H 表示，因此不会排除合法点；筛选结束又保证每个非法点至少违反一个保留约束，故约简模型的二进制可行点集仍恰好为 V。

这里的约简是相对于二进制点集而言的：约简后的连续可行区域可能大于原凸包，且贪心选择不保证约束数量全局最少。差分传播时取 $V = V_{\mathrm{D}}$，线性传播时取 $V = V_{\mathrm{L}}$，分别计算 H 表示并独立进行筛选。

3.7 全解枚举与等价性验证

按照讲义的验证方法，模型建立后将目标函数设为常数 0，仅枚举满足约束的全部二进制解，不按概率或相关度大小筛选。Gurobi 的 `PoolSearchMode=2` 用于系统搜索多个解，`PoolSolutions` 指定希望收集的解数。原始变量只有八位，取值空间为 $\{0,1\}^8$；对于上述四种模型，one-hot 的选择变量由合法点唯一确定，其余模型不含辅助变量，因此模型的不同完整解也不超过原始八位点的取值总数。将解池容量设为严格大于这一上界，并在求解以 `OPTIMAL` 状态结束且解池未满时，才确认全部解已收集。超时或中断时只记录已有解，不据此声称枚举完成。

读取解池时，逐个设置 `SolutionNumber`，再获取该解中各原始变量的值。讲义使用 `Xn` 属性，新版本 Gurobi 也提供 `PoolNX` 属性。对 one-hot 模型，还应核对恰有一个选择变量取 1，且该选择变量对应的点与原始八位变量相同。

记枚举结果在原始变量上的投影集合为 S_{enum}，则分别计算

$$V \setminus S_{\text{enum}}, \qquad S_{\text{enum}} \setminus V.$$

前者表示漏掉的合法点，后者表示多出的非法点。只有枚举已经完成且两个差集均为空，才说明模型的二进制解恰好为 V。差分与线性两类点集需要分别进行验证；解数相同只是必要条件，还需逐点比较。实际约束系数、条数、枚举结果及求解状态在实验内容部分记录。

4 实验内容

4.1 任务一：DDT 与 LAT 的建模及求解

本任务针对 CipherFour 的 4 比特 S 盒，分别采用 one-hot 法、逐点排除法、逻辑化简法和凸包法建立混合整数线性规划模型，并枚举各模型的全部二进制解。首先直接计算 S 盒的 DDT 和 LAT，再以表中的非零位置为参照，检查四种模型得到的输入输出对是否一致。DDT 中的 (a, b) 表示输入差分和输出差分，LAT 中的 (a, b) 表示输入掩码和输出掩码。

公共 DDT、LAT 的计算 四种方法共用 `ddt_lat.py` 中的表格计算函数。对于 DDT，遍历输入差分 a 和输入 x，计算 $S(x) \oplus S(x \oplus a)$，并在对应输出差分的位置累计计数。关键代码如下：

```
for a in range(SIZE):
    for x in range(SIZE):
        b = SBOX[x] ^ SBOX[x ^ a]
        table[a][b] += 1
```

对于 LAT，遍历输入、输出掩码 a, b，统计 $a \cdot x = b \cdot S(x)$ 成立的输入个数，再减去 8。程序通过按位与和 `bit_count()` 计算模 2 内积，其核心代码为：

```
equal_count = 0
for x in range(SIZE):
    input_parity = (a & x).bit_count() & 1
    output_parity = (b & SBOX[x]).bit_count() & 1
    if input_parity == output_parity:
        equal_count += 1
table[a][b] = equal_count - SIZE // 2
```

运行遍历计算得到图 1中的两张表。行下标表示输入，列下标表示输出，均使用十六进制。DDT 共有 72 个非零项，LAT 共有 122 个非零项。因此，差分模型应保留 72 个合法点，线性模型应保留 122 个合法点。

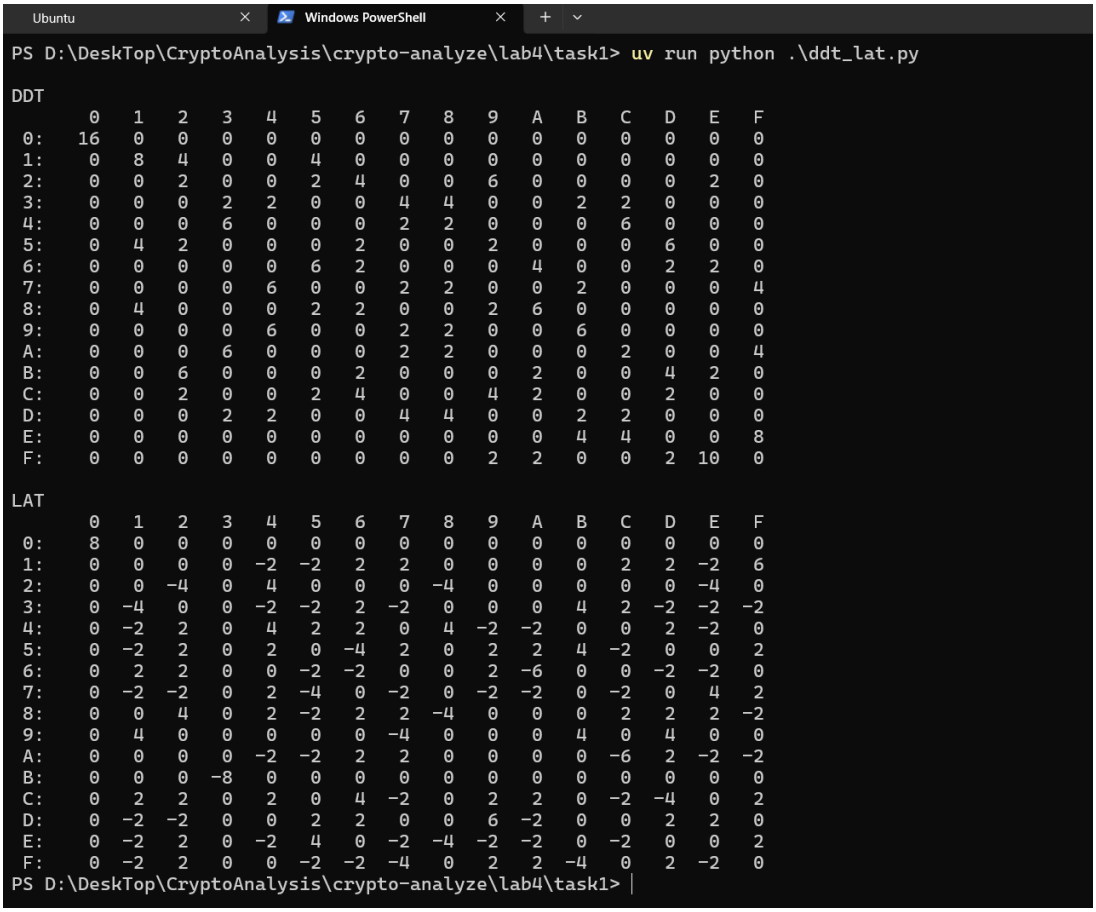


Figure 1: 直接遍历计算得到的 S 盒 DDT 与 LAT

统一的编码与枚举方式 程序将每个输入输出对编码为八位向量

$$z = (a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0),$$

其中输入、输出均按高位在前的顺序排列。四种方法使用相同的位序，以便直接对照求解结果。

为枚举所有合法点，各模型的目标函数均设为常数 0，并启用系统性解池搜索：

```
model.setObjective(0, gp.GRB.MINIMIZE)
model.Params.PoolSearchMode = 2
capacity = 2**8 + 1
model.Params.PoolSolutions = capacity
model.Params.MIPGap = 0
model.Params.MIPGapAbs = 0
model.optimize()
```

八位坐标共有 $2^8 = 256$ 种取值，合法点与完整变量赋值一一对应，因此将解池容量设为 257。程序在确认求解达到最优且解池未满后，逐个读取解池中的解，按统一位序还原输入输出，并以十六进制排序打印。每个解表示表中的一个非零位置，其具体计数或偏差仍由图 1给出。

4.1.1 one-hot 法

实验过程与关键代码 one-hot 法从 DDT 或 LAT 中提取所有非零位置，组成合法点列表 points。随后对每个合法输入输出对设置一个二进制选择变量 $\lambda_{a,b}$，并要求所有选择变量之和为 1。这样，每次求解都恰好选中一个合法点。

在 one_hot.py 中，先用公共函数 bits(a,b) 保存各合法点的八位编码，再建立 8 个原始变量和相应的选择变量：

```
points = [(a, b) for a in range(SIZE) for b in range(SIZE)
          if table[a][b] != 0]
point_bits = {point: bits(*point) for point in points}

z = model.addVars(8, vtype=gp.GRB.BINARY, name="z")
lam = model.addVars(points, vtype=gp.GRB.BINARY, name="lambda")
model.addConstr(lam.sum() == 1)
```

选择变量确定合法点后，还需要使 z 的每一位等于该点的对应坐标。对于第 j 位，只累加该位为 1 的合法点所对应的选择变量：

```
for j in range(8):
    model.addConstr(z[j] == gp.quicksum(
        lam[point] for point in points
        if point_bits[point][j] == 1))
```

因此，模型包含 1 条选点等式和 8 条坐标恢复等式，共 9 条等式。由于合法点列表中的每个输入输出对互不相同，一组八位坐标也唯一确定被选中的 λ。建立约束后，使用前面的解池参数分别枚举 DDT 模型与 LAT 模型。

实验结果与分析 代码运行结果如图 2所示。



Figure 2: one-hot 法的 DDT、LAT 模型全解枚举结果

DDT 模型有 72 个选择变量，加上 8 个原始变量，共 $72+8=80$ 个二进制变量；LAT 模型对应 $122+8=130$ 个二进制变量。两者均有 9 条等式，与上述建模过程一致。求解得到的 DDT 解数为 72，LAT 解数为 122，按输入分组后与图 1各行的非零列逐项相同，说明合法点被完整枚举，零表项对应的点均被排除。

4.1.2 逐点排除法

实验过程与关键代码 逐点排除法先找出表中所有取值为 0 的位置，组成非法点列表 `forbidden`。模型只设置 8 个原始二进制变量，然后对每个非法点添加一条排除不等式。由于 LAT 中正值和负值都属于合法表项，这里统一使用 `table[a][b] == 0` 提取非法点。

对于非法点 q，当 $q_j = 0$ 时，z_j 表示当前点在第 j 位是否与它不同；当 $q_j = 1$ 时，则由 $1 - z_j$ 表示。因此，将这 8 项相加并要求结果至少为 1，就能保证当前点至少有一位不同于 q。实现代码如下：

```
forbidden = [(a, b) for a in range(SIZE) for b in range(SIZE)
             if table[a][b] == 0]
z = model.addVars(8, vtype=gp.GRB.BINARY, name="z")

for point in forbidden:
    q = bits(*point)
    model.addConstr(gp.quicksum(
        z[j] if q[j] == 0 else 1 - z[j]
        for j in range(8)
    ) >= 1)
```

每条不等式只排除对应的一个非法二进制点。添加全部约束后，剩余可行点就是表中的非零位置，再由 Gurobi 按统一参数进行枚举。

实验结果与分析 代码运行结果如图 3所示。

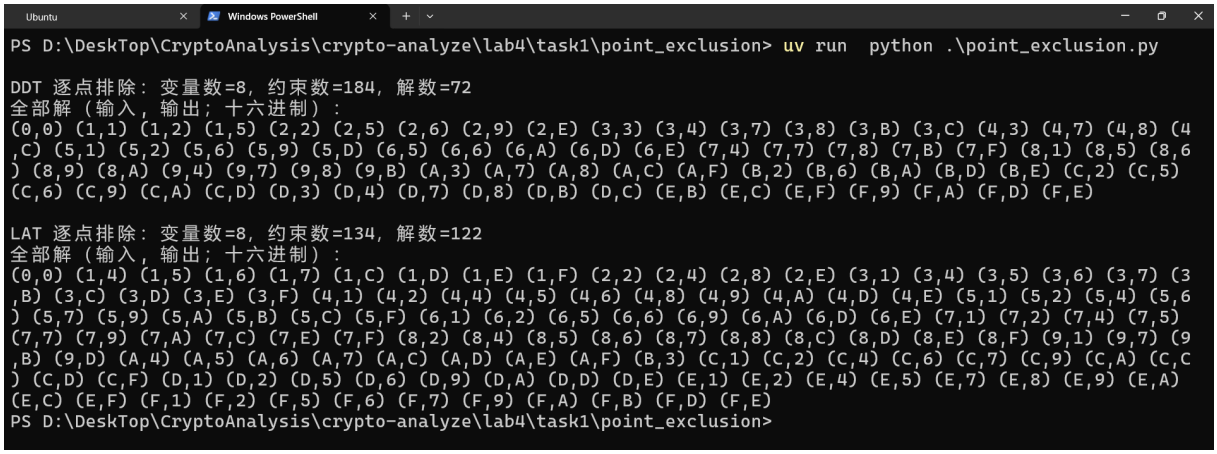


Figure 3: 逐点排除法的 DDT、LAT 模型全解枚举结果

输入、输出各有 16 种取值，因此共有 $16 \times 16 = 256$ 个候选点。DDT 中有 72 个合法点，需要排除 $256 - 72 = 184$ 个点，对应 184 条不等式；LAT 中有 122 个合法点，需要排除 $256 - 122 = 134$ 个点，对应 134 条不等式。两种模型均只有 8 个二进制变量，与程序输出一致。

枚举得到 72 个差分合法点和 122 个线性合法点，具体输入输出对与 one-hot 法及原始表的非零位置完全一致。该方法保持 8 个原始变量，通过 184 条或 134 条不等式逐点排除非法点，实现了相同的传播关系。

4.1.3 逻辑化简法

非法点函数的构造与化简 逐点排除法中，每个非法点都需要一条不等式。逻辑化简法进一步考虑将多个非法点合并处理：先构造一个八输入布尔函数，使其在非法点上取 1，在合法点上取 0，再使用 Espresso 将这个函数化简为若干乘积项。每个乘积项对应一个逻辑立方体，其中部分坐标固定为 0 或 1，其余坐标可以任意取值。

本实验在 `logic_minimization.py` 中通过 PyEDA 提供的 `espresso_exprs` 接口调用 Espresso。[1]程序按公共位序建立 8 个布尔变量，对每个零值位置生成一个最小项，再将所有最小项取或：

```
variables = exprvars("x", 8)
minterms = []
for a in range(SIZE):
    for b in range(SIZE):
        if table[a][b] == 0:
            point = bits(a, b)
            minterms.append(And(*(
                variable if value else ~variable
                for variable, value in zip(variables, point)
            )))

forbidden_function = Or(*minterms).to_dnf()
minimized, = espresso_exprs(forbidden_function)
```

化简后的析取范式仍表示同一组非法点。程序从 `minimized.to_dnf().cover` 中逐项读取文字：正文字表示该位固定为 1，负文字表示该位固定为 0，没有出现的变量用 `None` 表示通配位。这里根据文字的唯一标识找到原来的变量下标，使化简前后的坐标始终对应 $(a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0)$。

立方体到线性约束的转换 对于一个非法立方体，只要当前点有一个固定坐标与它不同，该点就位于立方体之外。因此，对固定为 0 的位置累加 z_j，对固定为 1 的位置累加 $1 - z_j$，并要求总和至少为 1。通配位不参与求和。关键代码为：

```
z = model.addVars(8, vtype=gp.GRB.BINARY, name="z")
for cube in cubes:
    model.addConstr(gp.quicksum(
        z[j] if value == 0 else 1 - z[j]
        for j, value in enumerate(cube) if value is not None
    ) >= 1)
```

这样，一条不等式可以排除一个立方体覆盖的多个非法点。Espresso 负责构造较紧凑的非法点覆盖，Gurobi 负责求解转换后的线性约束，两者在程序中依次执行。模型仍只有 8 个原始二进制变量。

[1]PyEDA 的 Espresso 接口及两层逻辑化简说明：https://pyeda.readthedocs.io/en/latest/2llm.html。

实验结果与分析 代码运行结果如图 4所示。



Figure 4: Espresso 逻辑化简后由 Gurobi 枚举得到的结果

本次 Espresso 调用为 DDT 非法点集生成 36 个立方体，为 LAT 非法点集生成 38 个立方体，因此两个模型分别包含 36 条和 38 条不等式。相比逐点排除法的 184 条和 134 条，逻辑化简在保留 8 个变量的同时减少了约束数量。

Gurobi 分别枚举出 72 个差分合法点和 122 个线性合法点，具体输入输出对与前两种方法一致，也与 DDT、LAT 中的非零位置一致。这说明立方体合并及其线性约束转换保持了原有的二进制解集。本实验使用的是 PyEDA 中的 Espresso 化简接口，36 和 38 记录本次得到的覆盖规模，不能据此认定约束数量已经达到全局最少。

4.1.4 凸包法

使用 SageMath 建立完整凸包 凸包法先从 DDT、LAT 的非零位置提取合法点，在八维实数空间中构造其凸包，再把凸包的 H 表示转换成线性约束。实验分为建立凸包和求解两个步骤：`build_hull.py` 在 WSL Arch 中使用 SageMath 计算约束，`convex_hull.py` 在 Windows 的 uv 环境中读取约束并调用 Gurobi。

建立凸包时，程序将合法输入输出对编码为八位向量，使用 `QQ` 进行精确有理数计算，并分别提取不等式和等式：

```
points = [bits(a, b) for a in range(SIZE) for b in range(SIZE)
          if table[a][b] != 0]
hull = Polyhedron(vertices=points, base_ring=QQ)

hrep = {
    "inequalities": sorted(integer_row(row)
                           for row in hull.inequalities_list()),
    "equations": sorted(integer_row(row)
                        for row in hull.equations_list()),
}
```

Sage 输出的每行系数按 $[c_0, c_1, \ldots, c_8]$ 排列，第一项为常数，对应 $c_0 + \sum_{j=0}^{7} c_{j+1} z_j \geq 0$，等式使用相同的系数顺序。[2]为便于 JSON 保存和后续整数计算，`integer_row()` 先乘以各系数分母的最小公倍数，再约去整数系数的最大公因数。整个过程使用正数缩放，保持不等式方向。

两组完整 H 表示保存在 `hull_constraints.json` 中，顶层键为 `DDT` 和 `LAT`，各自包含 `inequalities` 与 `equations` 两个系数数组。由于文件位于 Windows 工作目录，WSL 通过`/mnt/d/`访问并生成文件后，Windows 端可以直接读取相同数据。

贪心筛选与 Gurobi 建模 完整 H 表示可以准确描述凸包，但本任务最终只检查二进制点，因此进一步按每条不等式排除非法点的能力进行筛选。程序保留全部等式，将仍满足等式的非法点编号存入 `remaining`，再用 `pool` 记录每条候选不等式能够排除的点集。贪心选择的核心循环为：

```
while remaining:
    row = max(pool, key=lambda item: len(pool[item] & remaining),
              default=None)
    if row is None or not (pool[row] & remaining):
        raise RuntimeError("Incomplete illegal-point coverage")
    chosen.append(row)
    remaining.difference_update(pool.pop(row))
```

其中 `linear_form(row,point)` 计算该行系数在指定点上的值。值小于 0 表示点违反该不等式，因而可以被排除。候选系数按字典序排列，所以并列时选择顺序固定。每次保留的不等式均来自完整凸包，且筛选结束时所有非法点都已被排除，因此约简后的二进制可行点集仍与原合法点集一致。

最后建立 8 个二进制变量，将全部等式以 $h(z) = 0$、筛选后的不等式以 $h(z) \geq 0$ 加入 Gurobi，再按统一解池设置枚举全部可行点。

[2]SageMath 多面体构造与 H 表示说明：https://doc.sagemath.org/html/en/reference/discrete_geometry/sage/geometry/polyhedron/constructor.html。

实验结果与分析 SageMath 计算完整凸包的输出如图 5所示。

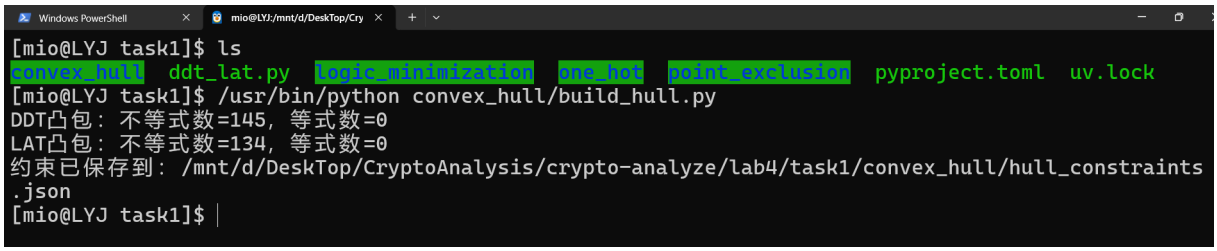


Figure 5: SageMath 建立 DDT、LAT 合法点集凸包并保存 H 表示

DDT 凸包共有 145 条不等式，LAT 凸包共有 134 条不等式，两者等式数均为 0。这与两组合法点集的仿射维数均为 8 相符。整数化后的约束与直接计算表进行核对时，完整 H 表示保留的二进制点分别为 72 个和 122 个，覆盖全部非零位置。

读取 JSON 并进行贪心筛选后，Gurobi 运行结果如图 6所示。



Figure 6: 凸包约束经贪心筛选后的 Gurobi 全解枚举结果

DDT 模型从 145 条不等式中保留 32 条，LAT 模型从 134 条中保留 28 条。由于没有等式，最终加入 Gurobi 的约束数分别就是 32 和 28，两种模型的变量数均为 8。枚举得到的 72 个差分输入输出对与 122 个线性掩码对，与前面三种方法及原始 DDT、LAT 的非零位置完全一致。

进一步对全部 256 个二进制点逐一代入约束，完整 H 表示和筛选后的约束均没有遗漏合法点，也没有保留非法点，说明约束读取、整数化和贪心筛选过程保持了二进制解集。这里的 32 条和 28 条约束用于描述合法二进制点；筛选后的连续可行区域可能大于完整凸包，贪心选择也不保证约束数量全局最少。

4.1.5 四种方法的结果对比

将四种方法的模型规模和枚举结果汇总，如表 1所示。表中的约束数统计显式添加的线性约束，二进制变量的取值范围单独计入变量类型；凸包法使用贪心筛选后实际加入 Gurobi 的约束数。

Table 1: 四种方法的模型规模与全解枚举结果

方法	DDT 模型			LAT 模型		
	变量数	约束数	解数	变量数	约束数	解数
one-hot 法	80	9	72	130	9	122
逐点排除法	8	184	72	8	134	122
逻辑化简法	8	36	72	8	38	122
凸包法	8	32	72	8	28	122

四种方法均准确复现 DDT 的 72 个与 LAT 的 122 个非零位置，说明二进制解集一致。one-hot 法通过增加选择变量减少线性约束条数，其 9 条约束均为等式；其余三种方法均使用 8 个变量，其中本次凸包贪心模型的约束数最少。表中反映的是模型规模，约束条数不能直接代替求解时间比较；Espresso 化简和贪心筛选的结果也不保证约束数量全局最少。